Projet de synthèse

Minh Hieu Nguyen

minh-hieu.nguyen@univ-lorraine.fr

Novembre MMXXVI

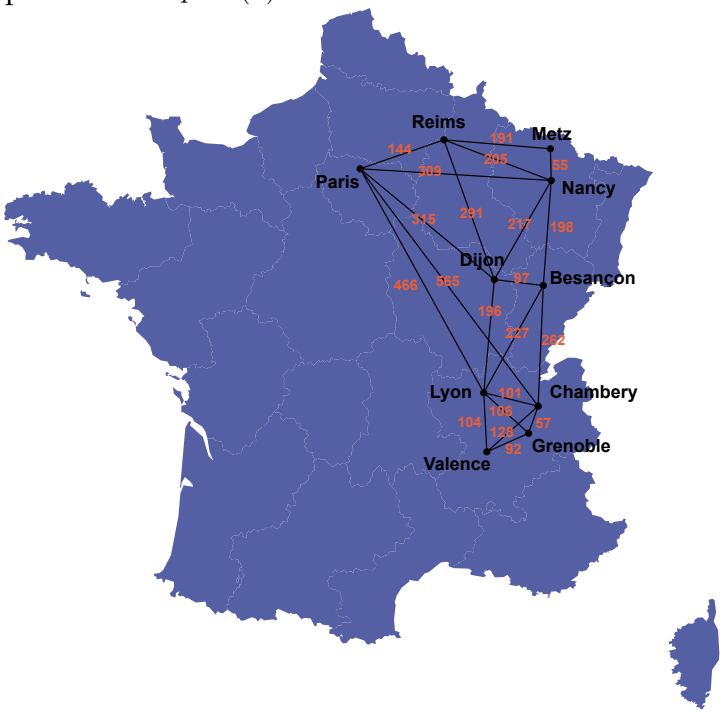
UNIVERSITÉ DE LORRAINE | UFR MATHÉMATIQUES INFORMATIQUE MÉCANIQUE ET AUTOMATIQUE

Le plus court chemin – Algorithme de Dijkstra

1 Introduction

L'objectif de ce projet est d'utiliser et d'appliquer les connaissances acquises au cours de l'année telles que Maths Discrètes, Programmation en C, Algorithmique et Structures de Données. Le projet va être réalisé en équipes de 2 personnes du même groupe TP.

Dans ce projet, vous allez implémenter l'algorithme proposé par Edsger Dijkstra en 1959 pour trouver *les plus court chemins* dans un graphe. Dans le problème du plus court chemin, l'entrée est un graphe simple non-orienté $G = (V, E)$, une fonction de pondération $w : E \to \mathbb{N}$ sur les arrêtes du graphe et un sommet $s \in V$ (appelé *départ*). Le poids d'un chemin entre deux sommets $u, v \in V$ est égal à la somme de poids des arrêtes qui forment ce chemin. L'objectif est de trouver le chemin de poids minimum (appelé *le plus court chemin*) entre le sommet de départ s et tout sommet $v \in V$, $v \neq s$. Pour chaque sommet $v \in V$, la sortie est le poids du plus court chemin $dist(v)$ (distance totale) de s à v, ainsi que son prédécesseur $pred(v)$ dans ce chemin.



Par exemple, ils existent plusieurs chemins pour aller de Metz à Lyon, passant par Dijon, par Besançon ou même par Paris. Par contre, le plus court chemin est Metz–Nancy–Dijon–Lyon de coût total égal à 468.

1.1 Structures de données

Dans ce projet on aura besoin d'implémenter plusieurs structures de données. D'abord, pour la représentation d'un graphe on utilise un *tableau* avec ses sommets. Ce tableau doit être de taille dynamique car on voudrait être capable d'ajouter ou de supprimer des sommets du graphe. En plus, chaque sommet doit stocker une *liste* avec les arêtes incidentes à lui.

L'algorithme de Dijkstra a besoin d'une *file de priorité* pour conserver les sommets qui ne sont pas encore étudiés dans l'ordre qu'il va les visiter. La priorité de chaque sommet correspond à sa distance actuelle depuis le sommet de départ : plus petite distance signifie plus grande priorité. Les primitives de cette file de priorité sont les suivantes :

insertion : ajouter un nouveau sommet à la file dans la bonne position (utile pour l'initialisation de la file de priorité).

extraction du sommet avec la plus grande priorité : retirer de la file le sommet avec la plus petite distance actuelle depuis le sommet de départ afin de traiter ses voisins.

incrémentation de la priorité d'un nœud donné : diminuer la distance actuelle depuis le sommet de départ (augmenter la priorité) d'un sommet donné.

Afin d'implémenter cette file de priorité, on va utiliser 3 différentes structures de données. Les structures de données qui nous intéressent sont les suivantes :

1. **liste doublement chaînée** (*double linked list*)
2. **arbre binaire complet** (*complete binary tree*) : un arbre binaire dans lequel tous les niveaux sont remplis à l'exception éventuelle du dernier, dans lequel les feuilles sont alignées à gauche (il doit être rempli de gauche à droit).
3. **tableau dynamique** (*dynamic table*) : un tableau dont la taille change en fonction du nombre d'éléments qui contient.

En conclusion, dans notre projet on va implémenter les structures de données suivantes :
— listes doublement chaînées,
— arbres binaires complets,
— tableaux dynamiques, et
— file de priorité :
  — tas basé sur un tableau dynamique,
  — tas basé sur des pointeurs (arbre binaire complet), et
  — listes ordonnées.

1.2 Évaluation de performance

L'objectif du projet n'est pas uniquement d'implémenter les algorithmes et les structures de données nécessaires mais aussi faire une évaluation de performance des différentes structures. Cette évaluation consiste à comparer leur performance au niveau théorique et pratique concernant la rapidité/efficacité d'exécution. Plus précisément, on s'intéresse à l'impact de l'implémentation d'une file de priorité avec une liste ordonnée ou un tas. Notre objectif est d'explorer si la différence de la performance est significative, si cette différence dépend de la taille de l'instance et si ça justifie les complexités théoriques asymptotiques.

1.3 Comment commencer ?

Pour vous aider à l'organisation de votre projet, le squelette de la partie en C est fourni sur Arche. Il contient 7 fichiers `.h` avec les prototypes des fonctions et les structures que vous devrez utiliser (répertoire `include\`) ainsi que 9 fichiers `.c` qui sont à modifier afin de réaliser le projet (répertoire `src\`).

Attention ! Vous n'avez pas le droit de modifier les prototypes des fonctions et des structures fournis. Autrement dit, vous ne devez pas modifier le nom des fonctions, le type et le nom des paramètres des fonctions et des structures, ainsi que leur nombre (vous ne pouvez pas ajouter ou supprimer des paramètres). En revanche, vous pouvez ajouter des fonctions auxiliaires.

Avant de commencer à coder, n'oubliez pas de créer un projet GIT par équipe (par exemple sur https://about.gitlab.com/ ou sur https://gitlab.univ-lorraine.fr/ en utilisant votre adresse email et votre mot de passe universitaire). Ajoutez y le squelette fourni. Ensuite, chaque membre de l'équipe peut avoir une copie locale du projet et commencer à travailler. N'oubliez pas de mettre à jour le serveur du GIT régulièrement (par exemple, une fois que le code d'une fonction sera finie et testée ou si un bogue sera corrigé), afin de communiquer à vos coéquipiers l'avancement que vous avez fait. *L'historique du GIT va faire partie de l'évaluation.*

Le document actuel donne une description des fichiers fournis et vous propose l'ordre d'implémentation du projet.

2 Fonctions auxiliaires fournies

Les fichiers `include/util.h` et `src/util.c` contiennent la définition et l'implémentation de certains éléments auxiliaires. D'abord, l'énumération suivante définit le type de la structure Les sous-programmes suivants sont fournies et vous aiderons dans la réalisation du projet.

```c
/* Renvoyer la valeur maximale parmi a et b. */
#define max(a,b) \
    ({ __typeof__ (a) _a = (a); \
        __typeof__ (b) _b = (b); \
        _a > _b ? _a : _b; })

/* Renvoyer la valeur minimale parmi a et b. */
#define min(a,b) \
    ({ __typeof__ (a) _a = (a); \
        __typeof__ (b) _b = (b); \
        _a < _b ? _a : _b; })

/* Affiche le message d'erreur msg et interrompt l'exécution
 * du programme si le paramètre interrupt vaut 1.*/
void ShowMessage(char * msg, int interrupt);

/* Afficher l'entier indiqué par le pointeur i. */
void viewInt(const void * i);

/* Libérer la mémoire de l'entier indiqué par le pointeur i. */
void freeInt(void * i);

/* Compare les entiers a et b et renvoie 1 si a<b, sinon 0. */
int intSmallerThan(const void * a, const void * b);

/* Compare les entiers a et b et renvoie 1 si a>b, sinon 0. */
int intGreaterThan(const void * a, const void * b);

/* Compare les entiers a et b et renvoie 1 si a=b, sinon 0. */
int intEqualTo(const void * a, const void * b);
```

3 Listes doublement chaînées

Vous avez déjà implémenté une liste doublement chaînée au cours "Programmation Avancée" du semestre précédent. Nous allons donc refaire cette partie et ajouter quelques fonctions supplémentaires pour commencer notre projet. En suite, vous trouverez une description des prototypes fournis ainsi que les fonctions que vous devez coder en forme d'exercices.

La structure suivante correspond à un nœud d'une liste (voir `include/list.h`) :

```
struct list_node_t {
    void * data;
    struct list_node_t * successor, * predecessor;
};
```

Notez que le type de données stockées dans chaque nœud de la liste est `void*`. Il s'agit d'un pointeur vers un élément dont le type n'est pas encore défini. La raison pour laquelle on veut avoir des données de type `void*` est d'éviter d'implémenter plusieurs listes chaînées, une pour chaque utilisation différente. Par exemple, si nous n'utilisions pas le type `void*`, nous devrions implémenter une liste pour stocker de valeurs entières, les arêtes, etc.

Exercice 1 Implémentez dans le fichier `src/list.c` les primitives de la structure correspondante au nœud d'une liste (les prototypes se trouvent dans `include/list.h`).

```
// Construction (n'oubliez pas d'allouer de la mémoire)
static struct list_node_t * new_list_node(void * data);

// Consultation
void * get_list_node_data(const struct list_node_t * node);
struct list_node_t * get_successor(const struct list_node_t * node);
struct list_node_t * get_predecessor(const struct list_node_t * node);

// Modification
void set_list_node_data(struct list_node_t * node, void * newData);
void set_successor(struct list_node_t * node, struct list_node_t *
   newSucc);
void set_predecessor(struct list_node_t * node, struct list_node_t *
   NewPred);
```

La structure suivante correspond à une liste (voir `include/list.h`) :

```
struct list_t {
    struct list_node_t * head, * tail;
    unsigned int size;
};
```

Exercice 2 Implémentez dans le fichier `src/list.c` les primitives de la structure d'une liste (les prototypes se trouvent dans `include/list.h`).

```
// Construction (n'oubliez pas d'allouer de la mémoire)
struct list_t * new_list();

// Consultation
int list_is_empty(struct list_t * L);
int get_list_size(const struct list_t * L);
struct list_node_t * get_list_head(const struct list_t * L);
```

```c
struct list_node_t * get_list_tail(const struct list_t * L);

// Modification
void increase_list_size(struct list_t * L);
void decrease_list_size(struct list_t * L);
void set_list_size(struct list_t * L, int newSize);
void set_list_head(struct list_t * L, struct list_node_t * newHead);
void set_list_tail(struct list_t * L, struct list_node_t * newTail);

// Affichage
void view_list(const struct list_t * L, void (*viewData)(const void *)
   );

// Libération de la mémoire
void delete_list(struct list_t * L, void (*freeData)(void *));
```

Exercice 3 Implémentez dans le fichier `src/list.c` les fonctions suivantes qui insèrent un élément dans la liste `L` (les prototypes se trouvent dans `include/list.h`). Faites attention de mettre à jour la taille de la liste lors de l'insertion d'un élément.

```c
/* Insère en tête de la liste L un nouveau nœud de donnée data. */
void list_insert_first(struct list_t * L, void * data);

/* Insère à la fin de la liste L un nouveau nœud de donnée data. */
void list_insert_last(struct list_t * L, void * data);

/* Insère un nouveau nœud de donnée data dans la liste L
 * après le nœud indiqué par le pointeur ptrelm. */
void list_insert_after(struct list_t * L, void * data, struct
   list_node_t * ptrelm);
```

Exercice 4 Implémentez dans le fichier `src/list.c` les fonctions suivantes qui suppriment un élément de la liste `L` (les prototypes se trouvent dans `include/list.h`). Faites attention de mettre à jour la taille de la liste lors de la suppression d'un élément.

```c
/* Supprime le premier nœud de la liste L et restitue sa donnée. */
void * list_remove_first(struct list_t * L);

/* Supprime le dernier nœud de la liste L et restitue sa donnée. */
void * list_remove_last(struct list_t * L);

/* Supprime le nœud de la liste L indiqué par le pointeur node
 * et restitue sa donnée. */
void * list_remove_node(struct list_t * L, struct list_node_t * node);
```

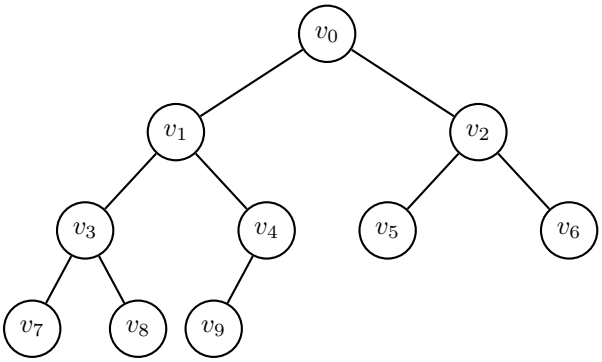
Exercice 5 Implémentez dans le fichier `src/list.c` les fonctions supplémentaires suivantes (les prototypes se trouvent dans `include/list.h`).

```c
/* Permute les données des deux nœuds en entrée. */
void list_swap_nodes_data(struct list_node_t * node1, struct
   list_node_t * node2);
```

```c
/* Vérifie si la donnée data existe déjà dans la liste. */
int list_data_exist(struct list_t * L, void * data);
```

4 Arbres binaires complets

Un arbre binaire complet (*complete binary tree* en anglais) est un arbre binaire dans lequel tous les niveaux sont remplis à l'exception éventuelle du dernier. En plus, au dernier niveau les feuilles sont alignées à gauche comme vous voyez sur la figure ci-dessous. Il faut toujours insérer un nouveau élément au dernier niveau, le plus à gauche possible. Si le dernier niveau est déjà rempli, on passe au niveau suivant (toujours le plus à gauche possible).



La structure suivante correspond à un nœud d'un arbre binaire (voir `include/tree.h`).

```c
struct tree_node_t {
    void * data; // la donnée
    struct tree_node_t * left   // le fils gauche
    struct tree_node_t * right; // le fils droit
};
```

Exercice 6 Implémentez dans le fichier `src/tree.c` les primitives de la structure correspondante au nœud d'un arbre binaire (les prototypes se trouvent dans `include/tree.h`).

```c
// Construction (n'oubliez pas d'allouer de la mémoire)
static struct tree_node_t * new_tree_node(void * data);

// Consultation
void * get_tree_node_data(const struct tree_node_t * node);
struct tree_node_t * get_left(const struct tree_node_t * node);
struct tree_node_t * get_right(const struct tree_node_t * node);

// Modification
void set_tree_node_data(struct tree_node_t * node, void * newData);
void set_left(struct tree_node_t * node, struct tree_node_t * newLeft)
   ;
void set_right(struct tree_node_t * node, struct tree_node_t *
   newRight);
```

La structure suivante correspond à un Arbre Binaire Complet (voir `include/tree.h`).

```c
struct tree_t {
    struct tree_node_t * root; // pointeur vers la racine
    unsigned int size; // nombre d'éléments
};
```

Exercice 7 Implémentez dans le fichier `src/tree.c` les primitives de la structure d'un arbre binaire complet (les prototypes se trouvent dans `include/tree.h`).

```c
// Construction (n'oubliez pas d'allouer de la mémoire)
struct tree_t * new_tree();

// Consultation
int tree_is_empty(const struct tree_t * T);
unsigned int get_tree_size(const struct tree_t * T);
struct tree_node_t * get_tree_root(const struct tree_t * T);

// Modification
void increase_tree_size(struct tree_t * T);
void decrease_tree_size(struct tree_t * T);
void set_tree_root(struct tree_t * T, struct tree_node_t * newRoot);

// Affichage
static void view_preorder(const struct tree_node_t * node, void (*
   viewData)(const void *));
static void view_inorder(const struct tree_node_t * node, void (*
   viewData)(const void *));
static void view_postorder(const struct tree_node_t * node, void (*
   viewData)(const void *));
void view_tree(const struct tree_t * T, void (*viewData)(const void *)
   , int order);

// Libération de la mémoire
static void free_subtree(struct tree_node_t * node, void (*freeData)(
   void *));
void delete_tree(struct tree_t * T, void (*freeData)(void *));
```

Notez que les fonctions `static` sont privées dans le fichier sur lequel elles sont déclarées (ici `src/tree.c`), c'est-à-dire nous ne pouvons pas les appeler dehors de ce fichier. Pour cette raison, on n'a pas besoin de définir leurs prototypes dans le fichier .h (ici `include/tree.h`).

Exercice 8 Implémentez dans le fichier `src/tree.c` les sous-programmes suivants qui réalisent l'insertion d'un nouveau élément dans un arbre binaire complet (les prototypes se trouvent dans `include/tree.h`). La fonction `insert_into_subtree` doit être implémentée de façon récursive, tandis que la procédure `tree_insert` doit lancer la récursion. Le paramètre `position` indique la première position disponible de l'arbre sur laquelle le nouveau élément va être inséré. Les positions de l'arbre sont numérotées par niveau et de gauche à droite.

```
/* Numérotation des positions d'un arbre binaire complet :
 *           0
 *         /   \
 *       1       2
 *      / \     / \
 *     3   4   5   6
 *    / \
 *   7  ...
 *
 * Insertion au dernier niveau et le plus à gauche possible
 * dans l'arbre T un nouveau nœud de donnée data */
static struct tree_node_t * insert_into_subtree(struct tree_node_t *
   node, unsigned int position, void * data);
void tree_insert(struct tree_t * T, void* data);
```

Exercice 9 Implémentez dans le fichier `src/tree.c` les fonctions suivantes qui suppriment un élément de l'arbre `T` (les prototypes se trouvent dans `include/tree.h`). La fonction `remove_from_subtree` doit être implémentée de façon récursive, tandis que la fonction `tree_remove` doit lancer la récursion. Le paramètre `position` indique la position du dernier élément de l'arbre, c'est-à-dire de l'élément qui va être supprimé.

```
/* Supprime le dernier nœud de l'arbre T et restitue sa donnée.
 * La mémoire du nœud supprimé est libérée mais pas la mémoire
 * de la donnée. */
static struct tree_node_t * remove_from_subtree(struct tree_node_t *
   node, unsigned int position, void** data);
void * tree_remove(struct tree_t * T);
```

Exercice 10 Implémentez dans le fichier `src/tree.c` les fonctions suivantes qui cherchent et restituent le nœud de l'arbre `T` qui se trouve à la position `position` (les prototypes se trouvent dans `include/tree.h`). La fonction `get_tree_node_at_position` doit être implémentée de façon récursive, tandis que la fonction `tree_find_node` doit lancer la récursion. Le paramètre `position` indique la position du nœud dans l'arbre, c'est-à-dire la position de l'élément qu'on cherche.

```
/* Restitue le nœud de l'arbre T qui se trouve à position. */
static struct tree_node_t * get_tree_node_at_position(struct
   tree_node_t * node, unsigned int position);
struct tree_node_t * tree_find_node(struct tree_t * T, unsigned int
   position);
```

Exercice 11 Implémentez dans le fichier `src/tree.c` la procédure suivante qui échange les données de deux nœud d'un arbre binaire complet donnés en paramètre (le prototype se trouve dans `include/tree.h`).

```
/* Permute les données des nœuds node1 et node2. */
void tree_swap_nodes_data(struct tree_node_t * node1, struct
   tree_node_t * node2);
```

5 Tableaux dynamiques

La taille d'un tableau classique en C est définie dans sa définition et n'est pas modifiable. Par contre, dans certaines applications nous avons besoin des tableaux de taille qui évolue pendant l'exécution de l'algorithme. Dans ce contexte, nous allons implémenter une structure de données pour représenter un tableau de taille dynamique. La taille va être toujours de forme 2^k, pour $k \in \mathbb{N}$. Pour de raisons de complexité, on impose que le tableau ait au moins taille$/4$ cases remplies.

La structure suivante correspond à un Tableau Dynamique (voir `include/dyntable.h`).

```
/* Le tableau dynamique est une structure contenant
 * (+) Un tableau (T) de pointeurs vers de données de type void,
 * (+) La taille (size) du tableau,
 * (+) Le nombre de cases actuellement utilisées (used) */
struct dyn_table_t {
    void **T;
    unsigned int size;
    unsigned int used;
};
```

Exercice 12 Implémentez dans le fichier `src/dyntable.c` les primitives de la structure d'un tableau dynamique (les prototypes se trouvent dans `include/dyntable.h`).

```
// Construction (n'oubliez pas d'allouer de la mémoire)
struct dyn_table_t * new_dyn_table();

// Consultation
void * get_dyn_table_data(const struct dyn_table_t * table, unsigned
   int position);
unsigned int get_dyn_table_size(const struct dyn_table_t * table);
unsigned int get_dyn_table_used(const struct dyn_table_t * table);
int dyn_table_is_full(const struct dyn_table_t * table);
int dyn_table_is_quasi_empty(const struct dyn_table_t * table);
int dyn_table_is_empty(const struct dyn_table_t * table);

// Modification
void set_dyn_table_data(struct dyn_table_t * table, unsigned int
   position, void * newData);
void set_dyn_table_size(struct dyn_table_t * table, unsigned int
   newSize);
void increase_dyn_table_used(struct dyn_table_t * table);
void decrease_dyn_table_used(struct dyn_table_t * table);

// Affichage
void view_dyn_table(const struct dyn_table_t * table, void (*viewData)
   (const void *));

// Libération de la mémoire
void delete_dyn_table(struct dyn_table_t * table, void (*freeData)(
   void *));
```

Exercice 13 Implémentez dans les fichier `src/dyntable.c` les procédures suivantes qui redimensionnent un tableau dynamique. Vous pouvez utiliser la fonction `realloc`. Notez que les deux fonctions sont définies comme `static` et on ne peut pas les appeler en dehors du fichier `src/dyntable.c`.

```
/* Dédoubler la taille du tableau table */
static void scale_up(struct dyn_table_t * table);

/* Diviser par 2 la taille du tableau table */
static void scale_down(struct dyn_table_t * table);
```

Exercice 14 Implémentez dans le fichier `src/dyntable.c` la procédure suivante qui réalise l'insertion d'un nouveau élément avec la donnée `data` dans la première case vide du tableau dynamique `table` (le prototype se trouve dans `include/dyntable.h`). Si le tableau est déjà plein, il faut utiliser la fonction `scale_up` avant l'insertion.

```
// Ajouter dans le tableau dynamique table un élément de donnée data.
void dyn_table_insert(struct dyn_table_t * table, void *data);
```

Exercice 15 Implémentez dans le fichier `src/dyntable.c` la fonction suivante qui réalise la suppression de l'élément qui se trouve dans la dernière case utilisée du tableau dynamique `table` (le prototype se trouve dans `include/dyntable.h`). La fonction renvoie la donnée de l'élément supprimé. Dans le cas où le nombre des cases remplies après la suppression est inférieur ou égale à `size/4`, n'oubliez pas de redimensionner le tableau en utilisant la fonction `scale_down`.

```
// Supprimer l'élément dans la position used-1 (dernier élément)
// du tableau dynamique table.
void * dyn_table_remove(struct dyn_table_t * table);
```

Exercice 16 Implémentez dans le fichier `src/dyntable.c` la procédure suivante qui échange le contenu des positions `pos1` et `pos2` du tableau dynamique `table` (le prototype se trouve dans `include/dyntable.h`).

```
// Permuter la donnée des positions pos1 et pos2 dans le tableau
// dynamique table.
void dyn_table_swap_nodes_data(struct dyn_table_t * table, unsigned
   int pos1, unsigned int pos2);
```

6 File de priorité - Tas minimier

En suite, nous allons implémenter trois différents *tas minimiers* basé sur les structures de données présentées ci-dessus. Un tas est appelé *minimier* si l'élément avec la plus grande priorité correspond à l'élément avec la plus petite clé (au contraire, dans un tas maximier l'élément avec la plus grande priorité correspond à l'élément avec la plus grande clé). Dans ce projet, la priorité sera définie par rapport à la distance actuelle entre le sommet de départ et le sommet en considération.

La structure suivante correspond à un nœud d'un tas (voir `include/heap.h`).

```
/* Un nœud du tas contient :
 * (+) une donnée (data),
 * (+) sa clé (key), et
 * (+) sa position au dictionnaire (dict_position).
```

```
 */
struct heap_node_t {
    unsigned long key;
    void * data;
    unsigned int dict_position;
};
```

Exercice 17 Implémentez dans le fichier `src/heap.c` les primitives de la structure correspondant au nœud d'un tas minimier (les prototypes se trouvent dans `include/heap.h`).

```
// Construction (n'oubliez pas d'allouer de la mémoire)
static struct heap_node_t * new_heap_node(unsigned long key, void *
   data);

// Consultation
unsigned long get_heap_node_key(const struct heap_node_t * node);
void * get_heap_node_data(const struct heap_node_t * node);
unsigned int get_heap_node_dict_position(const struct heap_node_t *
   node);

// Modification
void set_heap_node_key(struct heap_node_t * node, unsigned long newKey
   );
void set_heap_node_data(struct heap_node_t * node, void * newData);
void set_heap_node_dict_position(struct heap_node_t * node, unsigned
   int newPosition);
```

La structure suivante correspond à un Tas (voir `include/heap.h`). Comme expliqué ci-dessus, un tas est une structure de données pour implémenter une file de priorité et offre trois opérations (primitives) :

— *insertion* d'un élément dans le tas
— *extraction de l'élément avec la plus grande priorité* de tas
— *incrémentation de la priorité* d'un élément du tas

On va utiliser de pointeurs de fonctions pour définir de façon générique ces trois primitives, ainsi que les procédure d'affichage et de libération d'un tas et le test de vacuité.

```
/* Le tas est une structure qui contient les attributs suivants.
 * (+) La structure de données heap qui implémente le tas.
 * (+) Le dictionnaire dict : chaque élément du tas correspond à
 *     une position du dictionnaire qui ne change pas, même si la
 *     position de cet élément au tas peux changer. Il s'agit d'une
 *     référence fixe à l'extérieur du tas qui ne change pas quand le
 *     tas est réorganisé et qui contient la vraie position vers le
 *     tas de l'élément.
 *     Chaque case du dictionnaire contient un entier (int*) dans
 *     le cas où le tas est représenté par un tableau dynamique ou
 *     un arbre complet, et un nœud de liste (list_node_t*) dans
 *     le cas où le tas est représenté par une liste.
 * (+) Les pointeurs des 3 fonctions génériques implémentant les
 *     primitives du tas :
 *     (-) heap_insert
 *     (-) heap_extract_min
 *     (-) heap_increase_priority
```

```
 * (+) Les trois pointeurs de fonction suivants :
 *     (-) \p heap_is_empty : test de vacuité
 *     (-) \p view_heap : pour l'affichage
 *     (-) \p delete_heap : pour la suppression */
struct heap_t {
    void * heap;
    struct dyn_table_t * dict;
    unsigned int (*heap_insert)(struct heap_t * H, unsigned long key,
       void * data);
    struct heap_node_t * (*heap_extract_min)(struct heap_t * H);
    void (*heap_increase_priority)(struct heap_t * H, unsigned int
       dict_position, unsigned long newKey);
    int (*heap_is_empty)(const void * H);
    void (*view_heap)(const struct heap_t * H, void (*viewHeapNode)(
       const void *));
    void (*delete_heap)(struct heap_t * H, void (*freeHeapNode)(void
       *));
};
```

La fonction suivante crée et initialise un tas générique en spécifiant son type (son code est fourni dans `src/heap.c`).

```
// type =
//    0 (Dynamic Table Heap)
//    1 (Complete Binary Tree Heap)
//    2 (Ordered List Heap)
struct heap_t * new_heap(int type) {
    struct heap_t * H = calloc(1, sizeof(struct heap_t));
    if (H == NULL) {
        ShowMessage("heap:new_heap : The memory is full", 1);
        return NULL;
    }
    H->dict = new_dyn_table();
    switch (type) {
        case 0:
            H->heap = new_dyn_table();
            H->heap_insert = dyn_table_heap_insert;
            H->heap_extract_min = dyn_table_heap_extract_min;
            H->heap_increase_priority =
               dyn_table_heap_increase_priority;
            H->heap_is_empty = dyn_table_heap_is_empty;
            H->view_heap = view_dyn_table_heap;
            H->delete_heap = delete_dyn_table_heap;
            break;
        case 1:
            H->heap = new_tree();
            H->heap_insert = tree_heap_insert;
            H->heap_extract_min = tree_heap_extract_min;
            H->heap_increase_priority = tree_heap_increase_priority;
            H->heap_is_empty = tree_heap_is_empty;
            H->view_heap = view_tree_heap;
            H->delete_heap = delete_tree_heap;
            break;
```

```
        case 2:
            H->heap = new_list();
            H->heap_insert = list_heap_insert;
            H->heap_extract_min = list_heap_extract_min;
            H->heap_increase_priority = list_heap_increase_priority;
            H->heap_is_empty = list_heap_is_empty;
            H->view_heap = view_list_heap;
            H->delete_heap = delete_list_heap;
            break;
        default:
            ShowMessage("heap:new_heap : Unknown heap type", 1);
            return NULL;
    }
    return H;
}
```

En suite, on va implémenter les six primitives d'un tas pour chacun de trois types différents de tas introduits ci-dessus.

6.1 Implémentation par de listes ordonnées

Une liste ordonnée (*ordered list*) est une liste doublement chaînée, dont les éléments sont triés par ordre de priorité décroissant. L'élément avec la plus grande priorité / plus petite clé se trouve dans la première position de la liste (par exemple, dans notre application, le sommet avec la plus petite distance actuelle depuis le sommet de départ).

Exercice 18 Implémenter dans le fichier `src/heap.c` la fonction suivante qui réalise l'insertion d'un nouveau élément de donnée `data` et priorité `key` dans le tas `H` représenté par une liste ordonnée (le prototype se trouve dans `include/heap.h`). Le nouveau élément doit être placé dans la bonne position. A utiliser la procédure `list_insert_after`. N'oubliez pas à mettre à jour le dictionnaire, si besoin !

```
// Insérer dans le tas H un élément de donnée data et priorité key.
unsigned int list_heap_insert(struct heap_t * H, unsigned long key,
   void * data);
```

Exercice 19 Implémenter dans le fichier `src/heap.c` la fonction suivante qui extrait l'élément avec la plus grande priorité du tas `H` représenté par une liste ordonnée (le prototype se trouve dans `include/heap.h`). La fonction renvoie la donnée de l'élément extrait. N'oubliez pas à mettre à jour le dictionnaire, si besoin !

```
// Extraire du tas H la donnée du nœud avec la plus grande priorité.
struct heap_node_t * list_heap_extract_min(struct heap_t * H);
```

Exercice 20 Implémenter dans le fichier `src/heap.c` la procédure suivante qui augmente à `clé` la priorité du nœud qui se trouve à la position `dict_position` du dictionnaire dans le tas `H` représenté par une liste ordonnée (le prototype se trouve dans `include/heap.h`). A utiliser la procédure `list_swap_nodes_data`. N'oubliez pas à mettre à jour le dictionnaire, si besoin !

```
// Dans le tas H, incrémenter à clé la priorité du nœud
// correspondant à la position dict_position du dictionnaire.
void list_heap_increase_priority(struct heap_t * H, unsigned int
   dict_position, unsigned long newKey);
```

Exercice 21 Implémenter dans le fichier `src/heap.c` les sous-programmes suivants qui implémentent des fonctionnalités supplémentaires d'un tas `H` représenté par une liste ordonnée (les prototypes se trouvent dans `include/heap.h`).

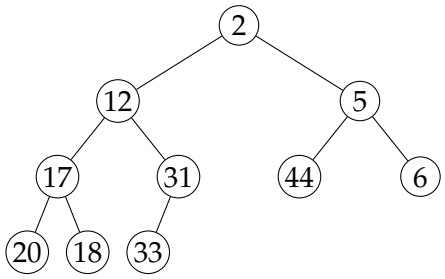
```
// Test de vacuité du tas H.
int list_heap_is_empty(const void * H);

// Afficher les éléments du tas H.
// Chaque élément est affiché grace au pointeur de fonction
// viewHeapNode.
void view_list_heap(const struct heap_t * H, void (*viewHeapNode)(
   const void *));

// Libérer la mémoire du tas H.
// Chaque élément est libéré en utilisant le pointeur de fonction
// freeHeapNode.
void delete_list_heap(struct heap_t * H, void (*freeHeapNode)(void *))
   ;
```

6.2 Implémentation par un arbre binaire complet ou par un tableau dynamique

Un arbre binaire complet peut être représenté par un tableau, sans avoir besoin d'une structure avec des nœuds et des pointeurs (fils gauche/droit). Une telle représentation est donnée dans la figure ci-dessous où l'arbre binaire complet est transformé à tableau en le parcourant niveau par niveau et de gauche à droite. Pour le nœud de la position i du tableau, son fils gauche se trouve dans la position $2i + 1$, son fils droit dans la position $2i + 2$ et son père dans la position $(i - 1)/2$.



0	1	2	3	4	5	6	7	8	9
2	12	5	17	31	44	6	20	18	33

On va alors utiliser les deux représentations équivalentes afin d'implémenter deux différentes structures de tas. Dans un tas minimier basé sur ces deux représentations, la propriété suivante est toujours vraie : pour tout nœud du tas, sa clé est plus petite ou égale que les clés situées dans ses fils gauche et droit. Autrement dit, l'élément avec la plus grande priorité se trouve toujours dans la racine, et chaque chemin de la racine vers une feuille quelconque est ordonné par ordre croissant des clés.

L'idée principale dans l'implémentation des primitives du tas est d'appliquer deux opérateurs de correction récursifs (vers le bas et vers le haut) après chaque modification du tas (ajout/suppression d'un élément ou modification de sa priorité). Pour plus d'information sur ces deux opérateurs, veuillez regarder la présentation du projet.

Exercice 22 Implémenter dans le fichier `src/heap.c` les deux opérateurs de correction d'un tas représenté par un tableau dynamique. N'oubliez pas à mettre à jour le dictionnaire, si besoin !

```
static void dyn_table_heap_update_upwards(struct heap_t * H, unsigned
   int position);

static void dyn_table_heap_update_downwards(struct heap_t * H,
   unsigned int position);
```

Exercice 23 Implémenter dans le fichier `src/heap.c` les trois primitives d'un tas `H` représenté par un tableau dynamique (les prototypes se trouvent dans `include/heap.h`). N'oubliez pas à mettre à jour le dictionnaire, si besoin !

```
// Insérer dans le tas H un élément de donnée data et priorité key.
unsigned int dyn_table_heap_insert(struct heap_t * H, unsigned long
   key, void * data);

// Extraire du tas H la donnée du nœud avec la plus grande priorité.
struct heap_node_t * dyn_table_heap_extract_min(struct heap_t * H);

// Dans le tas H, incrémenter à clé la priorité du nœud
// correspondant à la position dict_position du dictionnaire.
void dyn_table_heap_increase_priority(struct heap_t * H, unsigned int
   dict_position, unsigned long newKey);
```

Exercice 24 Implémenter dans le fichier `src/heap.c` les sous-programmes suivants qui implémentent des fonctionnalités supplémentaires d'un tas `H` représenté par un tableau dynamique (les prototypes se trouvent dans `include/heap.h`).

```
// Test de vacuité du tas H.
int dyn_table_heap_is_empty(const void * H);

// Afficher les éléments du tas H.
// Chaque élément est affiché grace au pointeur de fonction
// viewHeapNode.
void view_dyn_table_heap(const struct heap_t * H, void (*viewHeapNode)
   (const void *));

// Libérer la mémoire du tas H.
// Chaque élément est libéré en utilisant le pointeur de fonction
// freeHeapNode.
void delete_dyn_table_heap(struct heap_t * H, void (*freeHeapNode)(
   void *));
```

Exercice 25 Implémenter dans le fichier `src/heap.c` les deux opérateurs de correction d'un tas représenté par un arbre binaire complet. N'oubliez pas à mettre à jour le dictionnaire, si besoin !

```
// Procédure difficile !
static void tree_heap_update_upwards(struct heap_t * H, unsigned int
   position, struct tree_node_t * node);
```

```
static void tree_heap_update_downwards(struct heap_t * H, struct
    tree_node_t * node);
```

Exercice 26 Implémenter dans le fichier `src/heap.c` les trois primitives d'un tas `H` représenté par un arbre binaire complet (les prototypes se trouvent dans `include/heap.h`). N'oubliez pas à mettre à jour le dictionnaire, si besoin !

```
// Insérer dans le tas H un élément de donnée data et priorité key.
unsigned int tree_heap_insert(struct heap_t * H, unsigned long key,
    void * data);

// Extraire du tas H la donnée du nœud avec la plus grande priorité.
struct heap_node_t * tree_heap_extract_min(struct heap_t *H);

// Dans le tas H, incrémenter à clé la priorité du nœud
// correspondant à la position dict_position du dictionnaire.
void tree_heap_increase_priority(struct heap_t * H, unsigned int
    dict_position, unsigned long newKey);
```

Exercice 27 Implémenter dans le fichier `src/heap.c` les sous-programmes suivants qui implémentent des fonctionnalités supplémentaires d'un tas `H` représenté par un arbre binaire complet (les prototypes se trouve ntdans `include/heap.h`).

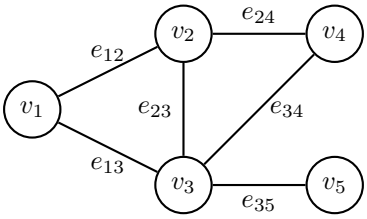
```
// Test de vacuité du tas H.
int tree_heap_is_empty(const void * H);

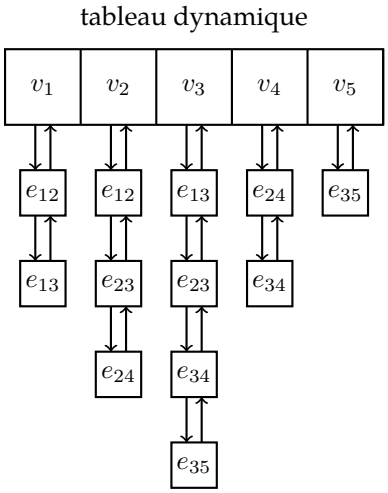
// Afficher les éléments du tas H.
// Chaque élément est affiché grace au pointeur de fonction
// viewHeapNode.
void view_tree_heap(const struct heap_t * H, void (*viewHeapNode)(
    const void *));

// Libérer la mémoire du tas H.
// Chaque élément est libéré en utilisant le pointeur de fonction
// freeHeapNode.
void delete_tree_heap(struct heap_t * H, void (*freeHeapNode)(void *))
    ;
```

7 Graphe

Le code pour la représentation d'un graphe est entièrement fourni. En suite, nous vous présentons brièvement les structures utilisées. Pour plus de détails, voir les fichiers `include/graph.h` et `src/graph.c`.





La figure ci-dessus donne un exemple de notre représentation. Dans cette représentation, on a besoin de définir les deux structures suivantes qui correspondent aux sommets et aux arêtes d'un graphe, respectivement.

```
// Un sommet (vertex_t) d'un graphe est une structure qui contient
// naturellement :
// (+) un identifiant (id), et
// (+) une liste avec les arêtes incidentes (incidence_list).
//
// Pour notre application de trouver le plus court chemin avec
// l'algorithme de Dijkstra, nous utiliserons aussi les attributs
// suivants :
// (+) total_distance : la distance à partir du nœud de depart
// (+) predecessor : le prédécesseur de ce sommet dans le plus court
//     chemin
// (+) dict_position : l'indice vers la position au dictionnaire
//     du tas qui contient ce sommet.
struct vertex_t {
    char * id;
    struct list_t * incidence_list;

    unsigned long total_distance;
    struct vertex_t * predecessor;
    unsigned int dict_position;
};

// Une arête E=(U,V) d'un graphe est caractérisée par ces deux
// extrémités U et V ainsi que par la distance entre ces deux sommets.
struct edge_t {
    struct vertex_t * U, * V;
    unsigned int distance;
};
```

Un graphe alors correspond à un tableau (dynamique) avec ses sommets. Plus précisément, on a la définition suivante :

```
typedef struct dyn_table_t * graph;
```

On peut retrouver les arêtes incidentes à chaque sommet dans la liste `incidence_list` de la structure `vertex`. Il n'y a pas alors une structure qui contient toutes les arêtes du graphe. Par contre, la

fonction `get_graph_edges` crée cette liste.

En dehors des primitives pour les trois structures (sommet, arête, graphe), nous vous donnons aussi une fonction qui peut construire un graphe enregistré dans un fichier. Le format de ce fichier est le suivant :

```
3            // nombre de sommets
SF           // id du sommet1
NY           // id du sommet2
LA           // id du sommet3
3            // nombre d'arêtes
SF NY 4683   // arête1: extrémité1 extrémité2 distance
SF LA 613    // ...
NY LA 4497   // ...
```

8 L'algorithme de Dijkstra

Vous avez maintenant tous les éléments nécessaires pour coder l'algorithme proposé par Dijkstra. Une description de l'algorithme est donnée en cours. Vous pouvez trouver la présentation sur Arche (diapo 14).

Exercice 28 Implémentez dans le fichier `src/algo.c` l'algorithme de Dijkstra (le prototype se trouve dans `include/algo.h`).

```
// Algorithme de Dijkstra
// G : un graphe
// source_name : nom du sommet de départ
// heap_type : type du tas
//             0--tableaux dynamiques
//             1--arbres binaires complets
//             2--listes ordonnées
void Dijkstra(graph G, const char * source_name, int heap_type);
```

Exercice 29 Implémentez dans le fichier `src/algo.c` les procédures suivantes qui vont afficher et sauvegarder, respectivement, une solution calculée par l'algorithme de Dijkstra (les prototypes se trouvent dans `include/algo.h`).

```
// Affichage de la solution de l'algorithme de Dijkstra
void view_solution(graph G, const char * source_name);

// Enregistrer dans un fichier la solution de l'algorithme de Dijkstra
void save_solution(const char * out_filename, graph G, const char *
   source_name);
```

Pour la dernière procédure, le format du fichier à créer est le suivant :

```
10              // nombre des sommets
Metz            // sommet de départ
Paris 335 Reims // sommet distance_totale prédécesseur
Lyon 468 Dijon  // ...
Nancy 55 Metz
Reims 191 Metz
Grenoble 572 Chambéry
```

```
Valence 572 Lyon
Chambéry 515 Besançon
Dijon 272 Nancy
Besançon 253 Nancy
```

9 Évaluation de performance d'algorithmes et de structures de données

Exercice 30 Réfléchissez sur la complexité asymptotique des primitives pour les trois implémentations du tas et remplissez le tableau suivant. La réponse à cette question doit apparaître à votre rapport avec une petite justification de quelques phrases (sans calculer le nombre d'instructions exact).

	insertion	extraction min	incrémentation de la priorité
liste ordonnée			
arbre binaire complet			
tableau dynamique			

Exercice 31 Ajoutez à votre `Makefile` une cible `expe` qui compile le code fourni dans le fichier `src/expe.c` et produit l'exécutable `expe` dans le répertoire `script`. Cet exécutable admet deux paramètres : le nom d'un fichier contenant un graphe et l'identifiant du sommet de départ. Le résultat de cet exécutable est une chaîne de caractères avec trois valeurs séparées par une espace. La première valeur correspond au temps d'exécution de l'algorithme de Dijkstra si on utilise un tas basé sur un tableau dynamique, la deuxième si on utilise un tas basé sur un arbre binaire complet et la troisième si on utilise un tas basé sur une liste ordonnée.

Exercice 32 Présentez dans votre rapport quelques résultats/comparaisons lesquels vous semblent intéressants. Vous pouvez créer quelques diagrammes qui comparent la performance en temps d'exécution de l'algorithme par rapport à la structure de données utilisées pour l'implémentation du tas. Quelle implémentation auriez vous choisi et pourquoi ? La réponse à cette question doit apparaître à votre rapport ainsi que la présentation des résultats de vos expériences.

10 Résultats attendus

- donner l'entrée du problème de plus court chemin (un graphe) :
  - choisir un fichier contenant le graphe
  - créer un graphe et enregistrer ce graphe dans un fichier
- saisir l'identifiant du sommet de départ parmi les sommets du graphe
- choisir le type de la structure pour le tas : 0–tableaux dynamique, 1–arbres binaires complets, 2–listes ordonnées
- appeler la fonction qui construit une solution de notre problème (algorithme de Dijkstra) implémentée en C avec les paramètres choisis (fichier d'entrée, sommet de départ, type de la structure) ainsi que le nom du fichier où la solution va être enregistrée
- afficher en gras les arrêtes qui participent aux plus courts chemins issus de la ville de départ ainsi que la distance optimale à coté de chaque ville

11 Dépôt et Évaluation

11.1 Le dépôt

Date limite : dimanche, 3 janvier, 23h59

1. le répertoire `.git/`
2. la totalité du code en C (fichiers .h et .c, Makefile, scripts) jusqu'à l'exercice 29 (inclus)

3. un README (facultatif) : comment lancer le programme C, explications divers, etc
4. un rapport de maximum 6 pages (format pdf)

Chaque binôme devra déposer une seule archive compressée (NOM1_NOM2.zip) contenant les éléments ci-dessus sur Arche. Aucune prolongation ne sera accordée : tout retard entraînera une pénalité d'un point (sur 20) par jour de retard.

11.2 Soutenance

Chaque équipe doit aussi présenter son projet pendant environ 30 minutes (questions incluses). Tous les membres de l'équipe participeront dans cette présentation. La note de cette présentation peut être différente pour les membres de la même équipe. Le programme des soutenances sera annoncé ultérieurement au cours du semestre.

11.3 Évaluation

Il n'y a pas d'examen terminal écrit.
— Utilisation du GIT :
— Utilisation des primitives :
— Listes doublement chaînées :
— Arbres binaires complets :
— Tableaux dynamiques :
— Tas :
— Algorithme de Dijkstra :
— Expériences :
— Rapport :
— Soutenance :

Il n'y aura pas d'épreuve de Seconde Chance.

11.4 Consignes pour le rapport

Le rapport final ne doit pas dépasser 6 pages en total. Le rapport n'est pas une documentation du code et doit impérativement contenir une introduction et une section des conclusions. Par exemple, vous pouvez expliquer :
— Quels outils vous avez utilisé ?
— Quelles fonctions supplémentaires vous avez introduit et pourquoi ?
— Quelle structure de données fonctionne le mieux en termes de temps d'exécution pour de petits instances ? Pour des instances moyennes ? Pour des grandes instances ? Pourquoi ?
— Quelles étaient les difficultés que vous avez rencontrées ?
— Quelles améliorations envisageriez vous ?
— Utilisation d'IA ?